

Lý thuyết chuyên sâu: Mảng tiền tố, Hậu tố & Mảng hiệu

Tác giả: Duc-Minh Vu | **Đơn vị:** SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA), Đại học Kinh tế Quốc dân (NEU)

Soạn thảo: Được hỗ trợ và soạn thảo bởi Agentic AI tool

Bản quyền © 2026 Duc-Minh Vu. Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.

1. Mảng tiền tố (Prefix Sum) & Mảng hậu tố (Suffix Sum) 1 chiều

1.1. Bản chất toán học

Cho dãy số $A = (A_1, A_2, \dots, A_N)$.

- **Mảng tiền tố (Prefix Sum)** P được định nghĩa là tổng tích lũy của các phần tử từ vị trí đầu tiên đến vị trí i :

$$P_i = \sum_{j=1}^i A_j \quad (P_0 = 0)$$

Công thức truy hồi xây dựng trong $O(N)$:

$$P_i = P_{i-1} + A_i \quad (\forall i = 1 \dots N)$$

- **Mảng hậu tố (Suffix Sum)** S được định nghĩa là tổng tích lũy từ vị trí i đến cuối dãy N :

$$S_i = \sum_{j=i}^N A_j \quad (S_{N+1} = 0)$$

Công thức truy hồi xây dựng trong $O(N)$:

$$S_i = S_{i+1} + A_i \quad (\forall i = N \dots 1)$$

1.2. Ứng dụng: Truy vấn tổng đoạn $[L, R]$ trong $O(1)$

Tổng các phần tử từ chỉ số L đến R ($1 \leq L \leq R \leq N$):

$$\sum_{j=L}^R A_j = \sum_{j=1}^R A_j - \sum_{j=1}^{L-1} A_j = P_R - P_{L-1}$$

- Thời gian tiền xử lý: $O(N)$.

- Thời gian mỗi truy vấn: $O(1)$.
- Không gian bộ nhớ: $O(N)$.

💡 **Quy tắc vàng 1-indexed:**

Luôn lưu mảng với chỉ số từ 1 đến N và đặt phần tử lính canh $P_0 = 0$. Khi đó, nếu $L = 1$, công thức $P_R - P_0 = P_R - 0 = P_R$ luôn tự động đúng mà không cần viết câu lệnh `if (L == 0)`.

2. Mảng hiệu 1 chiều (1D Difference Array)

2.1. Khái niệm & Mối quan hệ đối ngẫu

Cho mảng $A = (A_1, A_2, \dots, A_N)$. **Mảng hiệu (Difference Array) D** được định nghĩa là hiệu giữa hai phần tử liên tiếp:

$$D_1 = A_1, \quad D_i = A_i - A_{i-1} \quad (\forall i = 2 \dots N)$$

Tính chất đối ngẫu kỳ diệu:

Mảng cộng dồn của mảng hiệu chính là mảng ban đầu!

$$\sum_{j=1}^i D_j = D_1 + (A_2 - A_1) + (A_3 - A_2) + \dots + (A_i - A_{i-1}) = A_i$$

2.2. Kỹ thuật cập nhật đoạn $[L, R]$ với giá trị $+V$ trong $O(1)$

Giả sử ta cần cộng một lượng V vào tất cả các phần tử từ vị trí L đến vị trí R ($A_i \leftarrow A_i + V$ với mọi $i \in [L, R]$).

Xem xét sự thay đổi trên mảng hiệu D :

- Tại vị trí L : A_L tăng thêm V , trong khi A_{L-1} giữ nguyên $\implies D_L = A_L - A_{L-1}$ tăng thêm V .
- Tại các vị trí $i \in [L + 1, R]$: Cả A_i và A_{i-1} đều tăng thêm $V \implies D_i = A_i - A_{i-1}$ không đổi.
- Tại vị trí $R + 1$: A_{R+1} giữ nguyên, nhưng A_R tăng thêm $V \implies D_{R+1} = A_{R+1} - A_R$ giảm đi V .

Do đó, một thao tác cộng trên đoạn $[L, R]$ quy về đúng **2 thao tác điểm** trong $O(1)$:

$$\begin{aligned} D_L &\leftarrow D_L + V \\ D_{R+1} &\leftarrow D_{R+1} - V \quad (\text{nếu } R + 1 \leq N) \end{aligned}$$

- Sau khi thực hiện tất cả Q truy vấn cập nhật, ta tính mảng cộng dồn của D trong $O(N)$ để khôi phục lại trạng thái cuối cùng của mảng A .
- **Tổng độ phức tạp:** $O(N + Q)$ thay vì $O(N \times Q)$.

3. Mảng cộng dồn 2 chiều trên ma trận (2D Prefix Sum)

3.1. Định nghĩa

Cho ma trận kích thước $R \times C$. Mảng cộng dồn 2D $P[r][c]$ lưu tổng của tất cả các phần tử trong hình chữ nhật con từ góc trái trên $(1, 1)$ đến góc phải dưới (r, c) :

$$P[r][c] = \sum_{i=1}^r \sum_{j=1}^c A[i][j]$$

3.2. Công thức xây dựng theo Nguyên lý Bù trừ (Inclusion-Exclusion)

$$P[r][c] = P[r-1][c] + P[r][c-1] - P[r-1][c-1] + A[r][c]$$

- $P[r-1][c]$: Hình chữ nhật phía trên.
- $P[r][c-1]$: Hình chữ nhật bên trái.
- $P[r-1][c-1]$: Phần giao bị tính lặp 2 lần $\rightarrow$ trừ bớt 1 lần.
- $A[r][c]$: Giá trị của chính ô hiện tại.

3.3. Truy vấn tổng hình chữ nhật con bất kỳ trong $O(1)$

Tính tổng các ô trong hình chữ nhật có góc trái trên (r_1, c_1) và góc phải dưới (r_2, c_2) :

$$\text{Sum} = P[r_2][c_2] - P[r_1-1][c_2] - P[r_2][c_1-1] + P[r_1-1][c_1-1]$$

Thời gian truy vấn: $O(1)$.

4. Mảng hiệu 2 chiều (2D Difference Array)

Để cộng một lượng V vào tất cả các ô của hình chữ nhật con từ (r_1, c_1) đến (r_2, c_2) trong $O(1)$:

Ta chỉ cần thay đổi 4 góc trên mảng hiệu 2D D :

$$\begin{aligned} D[r_1][c_1] &\leftarrow D[r_1][c_1] + V \\ D[r_1][c_2+1] &\leftarrow D[r_1][c_2+1] - V \\ D[r_2+1][c_1] &\leftarrow D[r_2+1][c_1] - V \\ D[r_2+1][c_2+1] &\leftarrow D[r_2+1][c_2+1] + V \end{aligned}$$

Sau khi thực hiện xong tất cả các truy vấn cập nhật, ta áp dụng công thức mảng cộng dồn 2D trên mảng D để thu được ma trận kết quả:

$$D[r][c] = D[r-1][c] + D[r][c-1] - D[r-1][c-1] + D[r][c]$$

5. Các biến thể Tiền tố / Hậu tố đặc biệt

5.1. Prefix XOR (Mảng tiền tố phép XOR)

- Định nghĩa: $X_i = A_1 \oplus A_2 \oplus \dots \oplus A_i$.
- Vì phép XOR có tính chất tự triệt tiêu $x \oplus x = 0$, tổng XOR trên đoạn $[L, R]$ là:

$$\bigoplus_{j=L}^R A_j = X_R \oplus X_{L-1}$$

- Ứng dụng đếm đoạn con có tổng XOR bằng K :

$$X_R \oplus X_{L-1} = K \iff X_{L-1} = X_R \oplus K$$

Dùng bảng băm `std::unordered_map` hoặc mảng đếm tần số lưu số lần xuất hiện của X_{L-1} , giải bài toán trong $O(N)$!

5.2. Prefix GCD & Suffix GCD (Loại bỏ 1 phần tử)

Bài toán: Cho mảng N phần tử, tìm vị trí i cần loại bỏ sao cho gcd của $N - 1$ phần tử còn lại là lớn nhất.

- Ta tính trước hai mảng:
- $\text{prefGCD}[i] = \gcd(A_1, A_2, \dots, A_i)$
- $\text{suffGCD}[i] = \gcd(A_i, A_{i+1}, \dots, A_N)$
- Khi loại bỏ phần tử thứ i , gcd của các phần tử còn lại chính là:

$$\gcd(\text{prefGCD}[i-1], \text{suffGCD}[i+1])$$

Tính trong $O(1)$ (cộng thêm $O(\log(\text{val}))$ cho phép tính gcd).

5.3. Prefix Max & Suffix Max

- Dùng để xác định biên trái lớn nhất và biên phải lớn nhất của mỗi vị trí trong bài toán **Hứng nước mưa (Trapping Rain Water)**.
- Lượng nước giữ được tại vị trí i :

$$\text{water}[i] = \max(0, \min(\text{prefMax}[i], \text{suffMax}[i]) - A[i])$$

6. Tài liệu tham khảo & Bài giảng đề xuất (References)

6.1. Giáo trình & Sách thuật toán kinh điển

- Competitive Programmer's Handbook** — *Antti Laaksonen*:
- [Chapter 9: Range Queries \(Section 9.1: Static Array Queries & Prefix Sums\)](#).
- Guide to Competitive Programming (2nd Edition)** — *Antti Laaksonen*:

4. Chapter 4: Array Queries (Prefix sums, Difference arrays, Two-dimensional sums).
5. **Competitive Programming 4 (CP4 - Book 1)** — Steven Halim, Felix Halim:
6. Section 3.2: Direct Range Sum Queries and Grid Processing.

6.2. Bài giảng & Chuyên đề trực tuyến (Online Resources & Wikis)

1. **VNOI Wiki**:
 2. [Mảng cộng dồn và Mảng hiệu \(Prefix Sums & Difference Array\)](#). — Cẩm nang đầy đủ về kỹ thuật mảng cộng dồn 1D, 2D và mảng hiệu.
 3. **USACO Guide (Bronze & Silver Curriculum)**:
 4. [Introduction to Prefix Sums](#) — Lý thuyết trực quan, bài tập mẫu và phân loại dạng bài.
 5. [More on Prefix Sums \(2D Prefix Sums & Difference Arrays\)](#).
 6. **CP-Algorithms**:
 7. [Range Queries & Precomputation](#).
-
-



Competitive Programming Handbook

SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)

Tài liệu được soạn thảo và tối ưu bởi Agentic AI tool

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.